COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S2
DATE PERFORMED:	08/02/26	**DATE SUBMITTED:**	08/02/26
NAME:	Acala, Matt John C.	**INSTRUCTOR:**	Engr. Jimlord

ACTIVITY NO. 4 **Stacks**

1. PROCEDURE OUTPUT

B.2. Stacks using Linked Lists

Stack creation using Linked Lists

```
template <typename T>
class Node {
public:
    T data;
    Node<T>* next;
};

template <typename T>
class Stack {
private:
    Node<T> *head, *tail;

public:
    Stack() {
        head = NULL;
        tail = NULL;
    }
```

Push function

```
//adds a new element to the top of the stack
void push(T newData) {
    Node<T> *newNode = new Node<T>;
    newNode->data = newData;
    newNode->next = head;
    if (head == NULL) {
        head = tail = newNode;
    } else {
        newNode->next = head;
        head = newNode;
    }
}
```

Pop Function

```cpp
//removes and returns the top element of the stack
T pop() {
    T tempVal;
    Node<T> *temp;
    if (head == NULL) {
        std::cout << "Stack Underflow." << std::endl;
        return T(); // returns default value of type T (0, '\0', "" etc.)
    } else {
        temp = head;
        tempVal = temp->data;
        head = head->next;
        if (head == NULL) tail = NULL; // reset tail if stack becomes empty
        delete temp;
        return tempVal;
    }
}
```

Top Function

```cpp
//displays the current top element without removing it
void Top() {
    if (head == NULL) {
        std::cout << "Stack is Empty." << std::endl;
        return;
    } else {
        std::cout << "Top of Stack: " << head->data << std::endl;
    }
}
```

Traverse function to display all data/elements

```cpp
//traverses and prints all elements currently in the stack
void display() {
    if (head == NULL) {
        std::cout << "Stack is Empty." << std::endl;
        return;
    }
    std::cout << "Stack elements (top to bottom): ";
    Node<T> *current = head;
    while (current != NULL) {
        std::cout << current->data;
        if (current->next != NULL) std::cout << " -> ";
        current = current->next;
    }
    std::cout << std::endl;
}
```

Since its linked list we need a deconstructor to avoid memory leaks

```
    //cleans up remaining nodes to avoid memory leaks
    ~Stack() {
        while (head != NULL) {
            Node<T> *temp = head;
            head = head->next;
            delete temp;
        }
    }
};
```

MAIN

```
int main() {

    std::cout << "\n=== Char Stack ===" << std::endl;
    Stack<char> Stack;
    Stack.push('T');
    Stack.push('T');
    Stack.push('A');
    Stack.push('M');
    Stack.Top();
    Stack.display();
    Stack.pop();
    Stack.display();

    return 0;
}
```

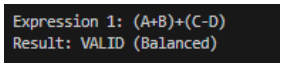
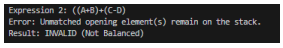
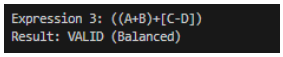
OUTPUT:

```
Top of Stack: M
Stack elements (top to bottom): M -> A -> T -> T
Stack elements (top to bottom): A -> T -> T
PS C:\Programming\C++\SCHOOL\CPE 010\HOA 4p1\Procedure>
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve problems using an implementation of stack:
The following problem definition and algorithm is provided for checking the balancing of symbols. Create an implementation using stacks. Your output must include the following:
a. Stack using Arrays
b. Stack using Linked Lists
c. (Optional) Stack using C++ STL
Problem Definition:
Stacks can be used to check whether the given expression has balanced symbols. This algorithm is very useful in compilers. Each time the parser reads one character at a time. If the character is an opening delimiter such as (, {, or [- then it is written to the stack. When a closing delimiter is encountered like), }, or]-the stack is popped. The opening and closing delimiters are then compared. If they match, the parsing of the string continues. If they do not

match, the parser indicates that there is an error on the line.
Steps:
1. Create a Stack.
2. While(end of input is not reached) {
a) If the character read is not a symbol to be balanced, ignore it.
b) If the character is an opening symbol, push it onto the stack.
c) If it is a closing symbol:
i) Report an error if the stack is empty.
ii) Otherwise, pop the stack.
d) If the symbol popped is not the corresponding opening symbol, report an error.
3. At the end of input, if the stack is not empty: report an error.
Self-Checking:
For the following cases, complete the table using the code you created.

Expression	Valid? (Y/N)	Output (Console Screenshot)	Analysis
(A+B) + (C-D)	Y		Every opening symbol has a matching closing symbol. Stack ends empty.
((A+B)+(C-D)	N		One (is never closed. Stack still has something left at the end.
((A+B)+[C-D])	Y		All symbols open and close in the right order. Stack ends empty.
((A+B]+[C-D]}	N		A] shows up but the last opened symbol was (, not [. Wrong pair.

T.I.P. TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES

COMPUTER ENGINEERING

QUEZON CITY

---

supplementaryLINKEDLIST.cpp

HOA 4p1 > Supplementary > supplementaryLINKEDLIST.cpp > LinkedStack > push(char)

```cpp
#include <iostream>
#include <string>

class Node {
public:
    char data;
    Node* next;
};

class LinkedStack {
private:
    Node *head, *tail;

public:
    LinkedStack() {
        head = NULL;
        tail = NULL;
    }

    bool isEmpty() {
        return head == NULL;
    }

    //creates a new node and places it at the head
    void push(char newData) {
        Node *newNode = new Node;
        newNode->data = newData;
        newNode->next = head;
        if (head == NULL) {
            head = tail = newNode;
        } else {
            head = newNode;
        }
    }

    //removes and returns the node at the head
    char pop() {
        if (isEmpty()) {
            std::cout << "Stack Underflow." << std::endl;
            return '\0';
        }
        Node *temp = head;
        char val = temp->data;
        head = head->next;
        if (head == NULL) tail = NULL;
        delete temp;
        return val;
    }

    char peek() {
        if (isEmpty()) return '\0';
        return head->data;
    }

    // Cleans up any remaining nodes
    ~LinkedStack() {
        while (head != NULL) {
            Node *temp = head;
            head = head->next;
            delete temp;
        }
    }
};

// Returns true if 'open' is the correct opening symbol for 'close'
bool isMatchingPair(char open, char close) {
    if (open == '(' && close == ')') return true;
    if (open == '{' && close == '}') return true;
    if (open == '[' && close == ']') return true;
    return false;
}

bool checkBalance(const std::string &expr) {
    LinkedStack stack;

    for (size_t i = 0; i < expr.length(); i++) {
        char ch = expr[i];

        // a) Ignore characters that are not elements to be balanced
        if (ch == '(' || ch == '{' || ch == '[') {
            // b) Opening element -> push onto stack
            stack.push(ch);
        }
        else if (ch == ')' || ch == '}' || ch == ']') {
            // c) Closing element found
            if (stack.isEmpty()) {
                // i) Report error if stack is empty
                std::cout << "Error: Closing element '" << ch
                          << "' found with no matching opening element (position "
                          << i << ")." << std::endl;
                return false;
            } else {
                // ii) Otherwise, pop the stack
                char popped = stack.pop();
                // d) If popped element doesn't correspond to closing element, report error
                if (!isMatchingPair(popped, ch)) {
                    std::cout << "Error: Mismatched elements. '" << popped
                              << "' does not match '" << ch << "' (position "
                              << i << ")." << std::endl;
                    return false;
                }
            }
        }
        // else: not a element to be balanced, ignore it
    }

    // 3. At the end of input, if stack is not empty, report an error
    if (!stack.isEmpty()) {
        std::cout << "Error: Unmatched opening element(s) remain on the stack."
                  << std::endl;
        return false;
    }

    return true;
}

int main() {
    std::string expressions[] = {
        "{A+B}+(C-D)",
```

supplementaryARRAY.cpp

HOA 4p1 > Supplementary > supplementaryARRAY.cpp > checkBalance(const std::string &)

```cpp
#include <iostream>
#include <string>
#include <cstring>

const int MAX_SIZE = 100;

class ArrayStack {
private:
    char items[MAX_SIZE];
    int top;

public:
    ArrayStack() {
        top = -1;
    }

    bool isEmpty() {
        return top == -1;
    }

    bool isFull() {
        return top == MAX_SIZE - 1;
    }

    //places a new element on top of the stack
    void push(char symbol) {
        if (isFull()) {
            std::cout << "Stack Overflow." << std::endl;
            return;
        }
        items[++top] = symbol;
    }

    //removes and returns the element at the top
    char pop() {
        if (isEmpty()) {
            std::cout << "Stack Underflow." << std::endl;
            return '\0';
        }
        return items[top--];
    }

    // Peek at the top element without removing it
    char peek() {
        if (isEmpty()) return '\0';
        return items[top];
    }
};

// Returns true if 'open' is the correct opening symbol for 'close'
bool isMatchingPair(char open, char close) {
    if (open == '(' && close == ')') return true;
    if (open == '{' && close == '}') return true;
    if (open == '[' && close == ']') return true;
    return false;
}

bool checkBalance(const std::string &expr) {
    ArrayStack stack;

    for (size_t i = 0; i < expr.length(); i++) {
        char ch = expr[i];

        // a) Ignore characters that are not elements to be balanced
        if (ch == '(' || ch == '{' || ch == '[') {
            // b) Opening element -> push onto stack
            stack.push(ch);
        }
        else if (ch == ')' || ch == '}' || ch == ']') {
            // c) Closing element found
            if (stack.isEmpty()) {
                // i) Report error if stack is empty
                std::cout << "Error: Closing element '" << ch
                          << "' found with no matching opening element (position "
                          << i << ")." << std::endl;
                return false;
            } else {
                // ii) Otherwise, pop the stack
                char popped = stack.pop();
                // d) If popped element doesn't correspond to closing element, report error
                if (!isMatchingPair(popped, ch)) {
                    std::cout << "Error: Mismatched elements. '" << popped
                              << "' does not match '" << ch << "' (position "
                              << i << ")." << std::endl;
                    return false;
                }
            }
        }
        // else: not a symbol to be balanced, ignore it
    }

    // 3. At the end of input, if stack is not empty, report an error
    if (!stack.isEmpty()) {
        std::cout << "Error: Unmatched opening symbol(s) remain on the stack."
                  << std::endl;
        return false;
    }

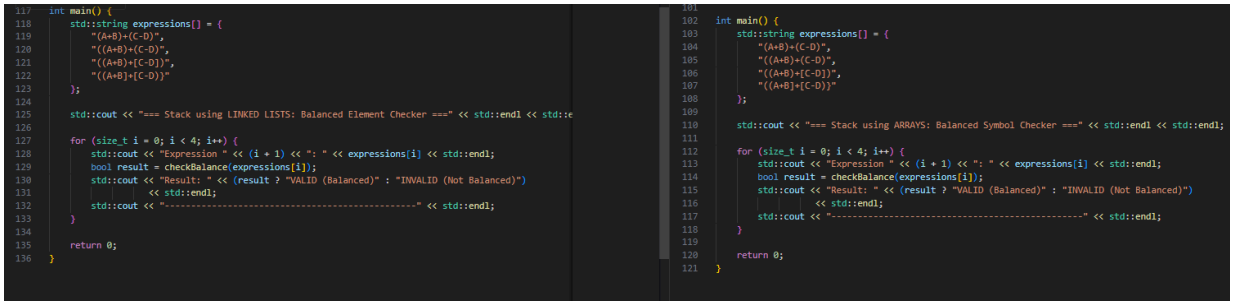
    return true;
}

int main() {
    std::string expressions[] = {
        "{A+B}+(C-D)",
        "((A+B)+(C-D)",
        "((A+B)+[C-D])",
        "((A+B]+[C-D))"
    };

    std::cout << "=== Stack using ARRAYS: Balanced Symbol Checker ===" << std::endl << std::endl;

    for (size_t i = 0; i < 4; i++) {
        std::cout << "Expression " << (i + 1) << ": " << expressions[i] << std::endl;
        bool result = checkBalance(expressions[i]);
        std::cout << "Result: " << (result ? "VALID (Balanced)" : "INVALID (Not Balanced)")
                  << std::endl;
        std::cout << "----------------------------------------------" << std::endl;
    }

    return 0;
}
```



```cpp
int main() {
    std::string expressions[] = {
        "(A+B)+(C-D)",
        "((A+B)+(C-D))",
        "((A+B)+[C-D])",
        "((A+B]+[C-D))"
    };

    std::cout << "=== Stack using LINKED LISTS: Balanced Element Checker ===" << std::endl << std::e

    for (size_t i = 0; i < 4; i++) {
        std::cout << "Expression " << (i + 1) << ": " << expressions[i] << std::endl;
        bool result = checkBalance(expressions[i]);
        std::cout << "Result: " << (result ? "VALID (Balanced)" : "INVALID (Not Balanced)")
                  << std::endl;
        std::cout << "-----------------------------------------------" << std::endl;
    }

    return 0;
}
```

```cpp
int main() {
    std::string expressions[] = {
        "(A+B)+(C-D)",
        "((A+B)+(C-D))",
        "((A+B)+[C-D])",
        "((A+B]+[C-D))"
    };

    std::cout << "=== Stack using ARRAYS: Balanced Symbol Checker ===" << std::endl << std::endl;

    for (size_t i = 0; i < 4; i++) {
        std::cout << "Expression " << (i + 1) << ": " << expressions[i] << std::endl;
        bool result = checkBalance(expressions[i]);
        std::cout << "Result: " << (result ? "VALID (Balanced)" : "INVALID (Not Balanced)")
                  << std::endl;
        std::cout << "-----------------------------------------------" << std::endl;
    }

    return 0;
}
```

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **CoPilot**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Idea generation, error explanator, and concept explanation**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.